

Zadanie indywidualne C#

Mediator + Fasada w świecie Gry o Tron

Krótki projekt konsolowy do wykonania samodzielnego

Czas wykonania	25–35 minut
Poziom	Średni
Technologia	C# / aplikacja konsolowa
Wzorce	Mediator + Fasada

1. Temat zadania

Zaprojektuj prosty system narady królewskiej inspirowany światem Gry o Tron. Różne rody zgłaszają sprawy do Korony, np. prośbę o sojusz, wezwanie do obrony zamku albo żądanie wysłania kruków. System ma pokazać współpracę dwóch wzorców projektowych: **Mediatora** oraz **Fasady**.

2. Cel

- Mediator ma koordynować komunikację między rodami i królewskimi usługami, tak aby obiekty nie wywoływały się bezpośrednio.
- Fasada ma udostępnić prosty punkt wejścia do całego procesu narady, np. jedną metodę `RozpocznijNarade(...)`.
- Program ma wypisać w konsoli przebieg obsługi co najmniej dwóch spraw zgłoszonych przez różne rody.

3. Opis sytuacji

W Westeros zbliża się kryzys. Do królewskiej rady wpływają prośby od rodów. Każda sprawa wymaga kilku działań: oceny przez doradcę, zapisania w kronice, wysłania kruka oraz ewentualnego wezwania wojsk. Nie chcemy, aby każdy ród znał wszystkie te klasy. Dlatego komunikację koordynuje Mediator. Z kolei Fasada ukrywa cały proces przed metodą `Main` i daje prosty interfejs uruchomienia narady.

4. Wymagane klasy i elementy

Model sprawy

- Utwórz klasę `CouncilRequest`.
- Powinna zawierać: `HouseName`, `RequestType`, `Description`, `IsUrgent`.
- Przykładowe typy spraw: `Alliance`, `Defense`, `RavenMessage`, `Supplies`.

Komponenty systemu

- `RoyalAdvisor` – ocenia sprawę i wypisuje krótką rekomendację.
- `RavenService` – wysyła kruka z wiadomością.
- `WarRoom` – reaguje na pilne sprawy obronne.
- `RoyalArchive` – zapisuje sprawę w kronice królestwa.

Mediator

- Utwórz interfejs `ICouncilMediator` z metodą np. `HandleRequest(CouncilRequest request)`.
- Utwórz klasę `CouncilMediator`, która zna komponenty systemu i decyduje, które z nich uruchomić.
- Mediator powinien koordynować komunikację między komponentami, a nie wykonywać całej pracy samodzielnie.

Fasada

- Utwórz klasę `KingdomCouncilFacade`.
- Fasada powinna tworzyć lub przyjmować mediatora i udostępniać prostą metodę, np. `StartCouncilSession(CouncilRequest request)`.
- Metoda `Main` powinna korzystać głównie z Fasady, a nie bezpośrednio ze wszystkich komponentów.

5. Oczekiwany sposób użycia w Main

W metodzie `Main` kod powinien wyglądać mniej więcej tak:

```
KingdomCouncilFacade council = new KingdomCouncilFacade();

CouncilRequest starkRequest = new CouncilRequest(
    houseName: "Stark",
    requestType: "Defense",
    description: "Prośba o wsparcie obrony Winterfell.",
    isUrgent: true);

CouncilRequest tyrellRequest = new CouncilRequest(
    houseName: "Tyrell",
    requestType: "Supplies",
    description: "Prośba o dostawę zboża dla stolicy.",
    isUrgent: false);

council.StartCouncilSession(starkRequest);
council.StartCouncilSession(tyrellRequest);
```

6. Przykładowy wynik działania

```
=== Narada królewska ===
Ród: Stark
Typ sprawy: Defense
Doradca: sprawa wymaga natychmiastowej uwagi.
WarRoom: przygotowano obronę zamku.
RavenService: wysłano kruka z odpowiedzią.
RoyalArchive: zapisano sprawę w kronice.

=== Narada królewska ===
Ród: Tyrell
Typ sprawy: Supplies
Doradca: sprawa standardowa.
RavenService: wysłano kruka z odpowiedzią.
RoyalArchive: zapisano sprawę w kronice.
```

7. Ograniczenia

- Nie umieszczaj całej logiki w Program.cs.
- Niech rody ani Main nie wywołują bezpośrednio wszystkich komponentów systemu.
- Mediator powinien koordynować komunikację między komponentami.
- Fasada powinna uprościć korzystanie z całego podsystemu.
- Projekt powinien być krótki i czytelny. Nie rozbudowuj go ponad potrzebę ćwiczenia.

8. Co należy oddać?

1. Projekt konsolowy C#.
2. Klasę CouncilRequest.
3. Komponenty: RoyalAdvisor, RavenService, WarRoom, RoyalArchive.
4. Interfejs ICouncilMediator i klasę CouncilMediator.
5. Klasę KingdomCouncilFacade.
6. Kod w Program.cs uruchamiający co najmniej dwie sprawy różnych rodów.
7. Krótkie odpowiedzi na pytania kontrolne.

9. Pytania kontrolne

1. Która klasa pełni rolę Mediatora?
2. Jaki problem rozwiązuje Mediator w tym zadaniu?
3. Która klasa pełni rolę Fasady?
4. Dlaczego Main powinien korzystać z Fasady zamiast bezpośrednio ze wszystkich komponentów?
5. Czym różni się Mediator od Fasady w tym przykładzie?
6. Co trzeba zmienić, jeśli dodamy nowy komponent, np. DragonService?
7. Czy komponent RoyalAdvisor powinien znać RavenService? Dlaczego?

10. Wariant dodatkowy dla szybszych osób

- Dodaj komponent DragonService, który reaguje tylko na bardzo pilne sprawy typu Defense.
- Dodaj trzecią sprawę od rodu Targaryen.
- Dodaj prostą historię narad, np. listę komunikatów zapisanych przez RoyalArchive.